Design Write up

Simer :

Routing state and route choice: The TinyOS distance-vector module keeps a fixed-size table of up to 32 routes. Each entry stores a destination, selected next hop, hop count, route state (valid or withdrawn), and last-update time. Every node starts with a self-route of cost 0. Neighbor Discovery supplies direct neighbors, each with cost 1. I used a stable current next hop on equal-cost updates.

Advertisements and Bellman-Ford: The DV Exchange module sends one-hop updates to confirmed neighbors roughly every 5-6 seconds. Each 28-byte packet has an 8-byte header and a 20-byte payload; with a count byte and two bytes per destination-cost pair, up to 9 routes fit in one packet. Larger vectors are split into multiple packets. Recipient-specific poison reverse advertises cost 16 when a route uses that recipient as the next hop. On receipt, our routing module calculates min(16, advertised cost + 1), accepts improvements, and also accepts worsening information from its current next hop. Cost 16 means unreachable.

Maintenance, withdrawal, and forwarding:-
Added 1-second maintenance timer refreshes direct-neighbor routes and checks selected routes for failed next hops. Learned routes expire after 20 seconds without a valid refresh. Unreachable routes are marked withdrawn, advertised with cost 16, and retained for 30 seconds before their slots are reclaimed, unless a new finite route is learned. The separate IP module looks up a usable next hop for unicast packets, decrements TTL at relays, and drops packets without a valid route instead of flooding. This design keeps route calculation separate from packet delivery.

Uday:

DV sending and packet checks: My DVExchange code gets the routing table and neighbor information from the other modules instead of keeping separate copies. It sends updates to each neighbor individually so poison reverse can advertise a cost of 16 when that neighbor is the next hop. If there are more than 9 routes, the remaining routes are sent in additional packets. Updates happen randomly every 5 to 6 seconds. I also added another timer that waits 150 ms between packets and retries after 250 ms if the queue is full. When a DV packet is received, the code checks that the packet is valid before sending the information to the routing table.

IP forwarding and delivery: For IP routing, I used the distance vector table to find the next hop for each packet. Instead of flooding the network, packets are sent directly to the next hop. When a node forwards a packet, it keeps the original source, destination, sequence number, and message the same, but decreases the TTL by 1. If the TTL is too low or there is no valid route, the packet is dropped. When a node receives a PING meant for itself, it creates a PINGREPLY and uses the routing table to send it back. I also updated the command handler to safely copy PING messages into the packet buffer.

Testing and implementation tradeoffs: We added six TOSSIM tests to check normal routing, alternate paths, network partitions, poison reverse, larger routing tables, and packet loss with TTL expiration. All six tests passed. One issue we had to account for was the limited sending queue, so I added a short delay between DV packets instead of trying to send everything at once. If the queue is full, the code retries rather than skipping the remaining routes. Radio packets can still get lost, which is why we continue sending periodic updates. We also used one byte for each destination ID to fit more routing entries into each packet.

Discussion Questions

1. Distance Vector vs. Link-State Routing

Distance Vector stores the destination, next hop, and cost. It exchanges routing information with direct neighbors. Link-State stores network topology information and floods updates across the network. Distance Vector is easier to implement but can converge more slowly. Link-State is more complex but generally converges faster.

2. Why accept a worse route from the current next hop?

Suppose Node 1 reaches Node 4 through Node 2 at cost 3. If Node 2 reports that the cost increased to 5, Node 1 must update its cost to 6. If it only accepts smaller costs, it keeps an outdated route and may forward packets incorrectly or create routing loops.

3. Split Horizon vs. Poison Reverse vs. Route Poisoning:

Suppose Node A reaches destination D through Node B.

- Split horizon: A does not advertise its route to D back to B.
- Poison reverse: A advertises D to B with cost 16 (unreachable).
- Route poisoning: After a route fails, the node advertises cost 16 to warn neighbors.

Poison reverse helps prevent two-node loops, but three or more nodes can still temporarily form loops using outdated routing information.

4. Are forward and reverse paths always symmetric?

No. Even when links are bidirectional, nodes independently choose their next hops. Equal-cost routes, network changes, and different convergence times can cause packets and replies to follow different paths.

5. What happens if a DV advertisement is lost or malformed?

Lost advertisements can be corrected by later periodic updates. Malformed packets are rejected using length, entry-count, and cost validation. Large vectors are split into multiple packets. If routing information is not refreshed within the timeout, the route becomes unreachable.

6. What if a node keeps changing routes or refuses to forward packets?

Repeated route advertisements and withdrawals can cause unstable routing. A node that advertises routes but does not forward packets can cause packet loss. One mitigation is to monitor forwarding reliability and avoid unreliable neighbors. However, this adds overhead and may mistake temporary packet loss for node failure.